COMP233 — Discrete Mathematics

Lecture 2.1 — Propositional Logic

Introduction and Basics

Ruwa' AbuHweidi @ 2026

Original course materials: Prof. Mustafa Jarrar

Updated course resources: COMP233 GitHub

Lecture 2.1 • The Logic of Compound Statements

What should you be able to do after this lecture?

Official scope: Epp §2.1 — Logical Form and Logical Equivalence

1 Recognize
Classify sentences as propositions or non-propositions and explain why.

2 Evaluate
Use negation, conjunction, inclusive disjunction, and XOR; construct and interpret truth tables.

3 Compare
Decide when statement forms are logically equivalent and detect nonequivalence with a counter-row.

4 Transform
Apply De Morgan's laws and the logical-equivalence laws to rewrite compound statements.

5 Classify & simplify
Recognize tautologies and contradictions, then simplify a statement form step by step.

Course objective connection: understand logical statements and avoid logical errors.

Why start discrete mathematics with logic?

From discrete structures to precise reasoning

Discrete mathematics
Studies objects that take distinct/separated values and objects that can be represented finitely or countably. It is a core mathematical language for computer science.

Logic
Studies methods of reasoning. It gives formal rules for judging arguments, expressing specifications, and reasoning about program behavior.

Algorithms | **AI / knowledge** | **Programming** | **Security** | **Software correctness**

The goal today is not merely to memorize symbols — it is to make the structure of reasoning explicit.

Propositions: the basic building blocks of logic

A proposition (statement) is a sentence that is true or false, but not both.

The definition concerns having a definite truth value — not whether the sentence happens to be true.

True proposition

"Amman is the capital of Jordan."

Truth value: T

True proposition

1 + 1 = 2

Truth value: T

False proposition

2 + 2 = 3

Truth value: F — but it is still a proposition.

Key distinction **"False" and "not a proposition" are different classifications.**

Not every sentence is a proposition

Ask: does the sentence have a definite truth value right now?

“What time is it?”
Question — it asks rather than asserts.

“Read this carefully.”
Command — not true or false.

x + 1 = 2
Open sentence — truth depends on x.

x + y = z
Open sentence — truth depends on x, y, z.

Context can matter
“He is a college student.” is not a proposition when “he” has no fixed referent. Once the referent is fixed, the sentence can have a definite truth value.

Quick classification
Which are propositions?
A. 5 is prime.
B. n is prime.
C. Please log in.
D. 9 < 2.

Answer mentally before moving on.

Negation: reversing a truth value

~p means "not p"

If p is true, ~p is false; if p is false, ~p is true.

p	~p
T	F
F	T

Language example

p: "The server is online."

~p: "The server is not online."

Precision check

The negation must express exactly what it means for p to be false — not merely a "negative-sounding" sentence.

Prediction: if p is false, what must ~(~p) be?

Conjunction: p ∧ q means “p and q”

p ∧ q is true exactly when both p and q are true.

Otherwise the conjunction is false.

p	q	p ∧ q
T	T	T
T	F	F
F	T	F
F	F	F

Computing example

p: “Password is correct.”
q: “MFA succeeds.”

Access requires both checks: p ∧ q.
Only the row (T,T) grants access.

Fast test: to make an AND statement false, is one false component enough?

Disjunction: p ∨ q means inclusive “p or q”

p ∨ q is true when at least one component is true.

It is false only when both p and q are false.

p	q	p ∨ q
T	T	T
T	F	T
F	T	T
F	F	F

Important distinction

Formal logic uses inclusive OR unless stated otherwise.

So p ∨ q is still true when p and q are both true.

If “exactly one” is intended, we use XOR (⊕).

What is the only row that makes p ∨ q false?

Exclusive OR (XOR): exactly one is true

p	q	p ⊕ q
T	T	F
T	F	T
F	T	T
F	F	F

$$p \oplus q \equiv (p \vee q) \wedge \sim(p \wedge q)$$

"At least one" AND "not both."

Inclusive OR

p ∨ q is true in three rows: TT, TF, FT.

Exclusive OR

p ⊕ q is true only in TF and FT.

Boundary case

When p = T and q = T, OR is true but XOR is false. This single row cleanly separates the two connectives.

Constructing truth tables systematically

Step 1
List all possible truth assignments for the statement variables.

Step 2
Create columns for useful subexpressions.

Step 3
Evaluate inner operations before the final connective.

Step 4
Interpret the final column, not just the symbols.

For n distinct statement variables, a complete truth table has 2^n rows.

p	q	p ∨ q	~p	(p ∨ q) ∧ ~p
T	T	T	F	F
T	F	T	F	F
F	T	T	T	T
F	F	F	T	F

Interpretation
The compound statement is true only when p = F and q = T.

A truth table is a reasoning model, not a spreadsheet

What logic does

Given the truth values of component propositions, logic determines the truth value of the compound statement.

What logic does not do

A truth table does not tell us whether the component facts are actually true in the real world; that evidence comes from elsewhere.

Example: access control

$$\text{password_correct} \wedge \text{mfa_success}$$

The logical rule combines two factual inputs.

Explain one row aloud:

If password_correct = T and mfa_success = F, then the conjunction is F: access is denied by this rule.

Logical equivalence: same behavior in every possible case

$P \equiv Q$

P and Q are logically equivalent iff they have identical truth values for every possible assignment to their statement variables.

Test 1

Construct truth tables for P and Q using the same variable assignments.

Test 2

Compare the final columns row by row.

Decision

All rows match → $P \equiv Q$.
One row differs → not equivalent.

To prove nonequivalence, one counter-row is enough.

Equivalent and non-equivalent forms side by side

Equivalent example

p	q	p ∧ q	q ∧ p
T	T	T	T
T	F	F	F
F	T	F	F
F	F	F	F

Final columns match in every row ⇒ p ∧ q ≡ q ∧ p.

Counterexample to equivalence

p	q	~(p ∧ q)	~p ∧ ~q
T	T	F	F
T	F	T	F
F	T	T	F
F	F	T	T

Rows TF and FT differ ⇒ the two forms are not logically equivalent.

Reasoning habit
When a proposed equivalence feels “obvious,” try to break it with a truth assignment before trusting it.

De Morgan’s laws: negation changes the connective

$\sim(p \wedge q) \equiv \sim p \vee \sim q$
Negation of AND → OR of the negations.

$\sim(p \vee q) \equiv \sim p \wedge \sim q$
Negation of OR → AND of the negations.

Example: conjunction
Original: “The server is online and authenticated.”

Negation: “The server is not online OR the server is not authenticated.”

Example: disjunction
Original: “The user is an admin or the user is the owner.”

Negation: “The user is not an admin AND the user is not the owner.”

A reliable verbal rule: negate each component and switch $\wedge$ ↔️ $\vee$.

Verify De Morgan with a truth table

p	q	~p	~q	p ∧ q	~(p ∧ q)	~p ∨ ~q
T	T	F	F	T	F	F
T	F	F	T	F	T	T
F	T	T	F	F	T	T
F	F	T	T	F	T	T

Compare the final two columns: they are identical in every row.

Therefore ~(p ∧ q) ≡ ~p ∨ ~q

Tautologies and contradictions: truth determined by form

Tautology
A statement form that is true for every possible truth assignment.

Contradiction
A statement form that is false for every possible truth assignment.

p	~p	p ∨ ~p
T	F	T
F	T	T

p ∨ ~p is always true.

p	~p	p ∧ ~p
T	F	F
F	T	F

p ∧ ~p is always false.

Logical equivalence laws — Part I

Let t be a tautology and c be a contradiction.

Law	∧ / first form	∨ / paired form
Commutative	$p \wedge q \equiv q \wedge p$	$p \vee q \equiv q \vee p$
Associative	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	$(p \vee q) \vee r \equiv p \vee (q \vee r)$
Distributive	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
Identity	$p \wedge t \equiv p$	$p \vee c \equiv p$
Negation	$p \vee \sim p \equiv t$	$p \wedge \sim p \equiv c$
Double negative	$\sim(\sim p) \equiv p$	

Use laws as justified rewrite rules — each step should name the law being applied.

Logical equivalence laws — Part II

Continue the same law bank; t = tautology, c = contradiction.

Law	First form	Paired form
Idempotent	$p \wedge p \equiv p$	$p \vee p \equiv p$
Universal bound	$p \wedge c \equiv c$	$p \vee t \equiv t$
De Morgan	$\sim(p \wedge q) \equiv \sim p \vee \sim q$	$\sim(p \vee q) \equiv \sim p \wedge \sim q$
Absorption	$p \wedge (p \vee q) \equiv p$	$p \vee (p \wedge q) \equiv p$
Negations of t,c	$\sim t \equiv c$	$\sim c \equiv t$

Strategy
Truth tables can always decide equivalence; algebraic laws are often faster when a useful pattern is visible.

Simplifying a statement form — with a reason for every step

$$\sim(\sim p \wedge q) \wedge (p \vee q) \equiv p$$

Original example from the lecture, reconstructed cleanly.

Equivalent form	Justification
$\sim(\sim p \wedge q) \wedge (p \vee q)$	Given
$(p \vee \sim q) \wedge (p \vee q)$	De Morgan's law
$p \vee (\sim q \wedge q)$	Distributive law
$p \vee (q \wedge \sim q)$	Commutative law for $\wedge$
$p \vee c$	Negation law
p	Identity law

A correct simplification is a chain of equivalences, not a sequence of guesses.

Why logical simplification matters in computing

Programs
Equivalent Boolean conditions can express the same decision with a clearer or more efficient condition.

Digital logic
Equivalent expressions can correspond to different gate-level implementations of the same Boolean behavior.

Reasoning
A simpler equivalent form can make a later argument or proof easier to see.

$$(p \wedge q) \vee (p \wedge \sim q) \equiv p \wedge (q \vee \sim q) \equiv p$$

The q-case split disappears because $q \vee \sim q$ is a tautology.

Think like a compiler or circuit designer: preserve behavior, simplify structure.

Concept check — decide before calculating

Use 4–5 minutes individually, then compare with a partner.

1. Is “x + 1 = 2” a proposition as written? Explain.

2. When is $p \vee q$ false?

3. Give one row that proves $p \vee q$ and $p \oplus q$ are not equivalent.

4. Negate “The account is active AND verified.”

5. Classify $p \wedge \sim p$: tautology, contradiction, or neither?

What this lecture prepares you for next

Today

Propositions
→ connectives
→ truth tables
→ equivalence
→ simplification



Next

If p, then q
p → q

When exactly is an "if–then" statement false?

Reasoning ahead

Converse
Inverse
Contrapositive
Biconditional

— all depend on today's truth-table discipline.

Learning Appendix

Use for revision, independent practice, and exam preparation.

Appendix A — Core concepts at a glance

Proposition
A sentence with a definite truth value: T or F.

Negation
~p has the opposite truth value of p.

Conjunction
p ∧ q is true only in TT.

Disjunction
p ∨ q is false only in FF; OR is inclusive.

XOR
p ⊕ q is true exactly in TF and FT.

Equivalence
P ≡ Q iff their final truth-table columns always match.

De Morgan
Negate each component and switch ∧ ↔ ∨.

Tautology / contradiction
All T rows / all F rows.

Appendix A — Common misconceptions and fast fixes

Misconception	Correction
A false sentence is not a proposition.	Wrong. False propositions still have definite truth value F.
“or” always means exactly one.	In formal logic, ∨ is inclusive. Use ⊕ for exactly one.
∼(p ∧ q) means ∼p ∧ ∼q.	Wrong. De Morgan gives ∼p ∨ ∼q.
Equivalent means “often the same.”	Equivalent means same truth value in every possible row.
One matching row proves equivalence.	No. All rows must match; one differing row disproves equivalence.

Diagnostic habit
When uncertain, translate precisely and test a carefully chosen truth assignment.

Appendix B — Conceptual connections



Special truth-table patterns

All T
TAUTOLOGY

All F
CONTRADICTION

Equivalent forms can be transformed with laws

Equivalence laws
↓
Simpler form

code / circuits / proofs

The dependency is structural: definitions create truth tables; truth tables support equivalence; equivalence laws support simplification.

Appendix C — Worked example 1: classify, then negate

Prompt

Classify each sentence as a proposition or not. If it is a proposition, write its negation.

A. 12 is divisible by 3.
B. What is your ID?
C. $x > 5$.
D. The account is locked.

Item	Classification	Reason / negation
A	Proposition (T)	12 is not divisible by 3.
B	Not a proposition	Question — no truth value.
C	Not a proposition	Open sentence; x is unspecified.
D	Proposition	The account is not locked.

Appendix C — Worked example 2: prove an equivalence by truth table

Show: $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

A distributive law for ∨ over ∧.

p	q	r	$p \vee (q \wedge r)$	$(p \vee q) \wedge (p \vee r)$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	T	T
F	T	T	T	T
F	T	F	F	F
F	F	T	F	F
F	F	F	F	F

The last two columns match in all 8 rows ⇒ the statement forms are logically equivalent.

Appendix C — Worked example 3: simplify algebraically

$$(p \wedge q) \vee (p \wedge \sim q) \equiv p$$

Equivalent form	Reason
$(p \wedge q) \vee (p \wedge \sim q)$	Given
$p \wedge (q \vee \sim q)$	Distributive law
$p \wedge t$	Negation law
p	Identity law

Why this is safe
Every arrow is an equivalence; therefore the simplified expression has exactly the same truth behavior as the original.

Appendix D — Knowledge check (1–4)

Do not look at the answer key until you commit to an answer.

1 MCQ • Easy • Understand

Which is NOT a proposition?

A. 7 is prime B. 2 + 2 = 5 C. x + 1 = 2 D. 9 < 4

2 True/False • Easy • Understand

If p = T and q = T, then p ∨ q = F. Justify your answer.

3 Error detection • Medium • Analyze

A student writes ~(p ∧ q) ≡ ~p ∧ ~q. Find a truth assignment that exposes the error.

4 Counterexample • Medium • Analyze

Show that p ∨ q is not logically equivalent to p ⊕ q using one row only.

Appendix D — Knowledge check (5–8)

5 Truth table • Medium • Apply
Construct the truth table for (p ∨ q) ∧ ~p. When is it true?

6 Classification • Medium • Analyze
Classify (p ∨ ~p) ∧ q as a tautology, contradiction, or neither. Justify.

7 Simplification • Medium • Apply
Simplify (p ∧ q) ∨ (p ∧ ~q) using named logical-equivalence laws.

8 Proof/disproof • Hard • Evaluate
Is ~(p ∨ q) logically equivalent to ~p ∨ ~q? Prove or disprove.

Appendix E — Answers and explanations (1–4)

1 Answer: C
$x + 1 = 2$ is open because x is unspecified. B and D are false propositions, but both still have definite truth values.

2 Answer: False
Inclusive OR is true when at least one component is true. In the TT row, $p \vee q = T$.

3 Counter-row
Take $p = T$, $q = F$. Then $\sim(p \wedge q) = T$, but $\sim p \wedge \sim q = F$. Therefore the proposed equivalence is false.

4 Counter-row
Take $p = T$, $q = T$. Then $p \vee q = T$, but $p \oplus q = F$. One differing row proves nonequivalence.

Common error
Do not confuse “false proposition” with “not a proposition,” and do not assume everyday OR means XOR.

Appendix E — Answers and explanations (5–8)

5 Truth table result
The final column is F, F, T, F for rows TT, TF, FT, FF. It is true only when p = F and q = T.

6 Neither
$(p \vee \sim p) \wedge q \equiv t \wedge q \equiv q$. Since q can be T or F, the form is neither a tautology nor a contradiction.

7 Simplification
$(p \wedge q) \vee (p \wedge \sim q) \equiv p \wedge (q \vee \sim q) \equiv p \wedge t \equiv p$.
Laws: distributive, negation, identity.

8 Disproof
No. De Morgan gives $\sim(p \vee q) \equiv \sim p \wedge \sim q$. Counter-row p = T, q = F: left F, proposed right T.

Reasoning standard
For “prove equivalence,” show all cases or use valid equivalence laws. For “disprove,” one counter-row is sufficient.

Appendix F — Optional enrichment: where propositional logic appears

Computer Engineering
AND / OR / NOT / XOR are the behavioral core of digital logic gates. Truth tables specify the input-output behavior of a circuit before implementation.

Cybersecurity
Access-control policies are Boolean conditions. De Morgan's laws are especially useful when reasoning about the negation of "allowed" conditions and deny rules.

AI / CS
Propositional facts and Boolean constraints appear in rule systems, SAT-style reasoning, configuration problems, and program conditions.

Same truth table ⇒ same Boolean behavior

That is the practical meaning of logical equivalence in these systems.

References and course resources

Primary textbook
Susanna S. Epp, Discrete Mathematics with Applications, 4th edition. Lecture 2.1 aligns with §2.1, “Logical Form and Logical Equivalence.”

Course scope
COMP233 — Discrete Mathematics, Birzeit University, First Semester 2026/2027. Chapter 2 covers §2.1–2.3.

Original course materials: Prof. Mustafa Jarrar

Updated course resources: COMP233 GitHub

Acknowledgment
The original lecture deck identifies Mustafa Jarrar & Radi Jarrar in the lecture-note authorship. Dr. Radi Jarrar is acknowledged here for that contribution.

Reconstructed and enhanced for instruction by Ruwa' AbuHweidi @ 2026